

SWE 645 – HW3 Complete Setup Guide

How I got the Student Survey app (React + FastAPI/SQLModel + MySQL) built, containerized, and running on my HW2 Kubernetes cluster

Overview

For HW2 I already had an EC2 instance running Rancher with a single-node Kubernetes cluster on it. Rather than spin up a second box for HW3, I decided to reuse that same instance — it's already got Docker, Rancher, and kubectl configured, so the only new tool I actually need is Helm.

The bigger change from how I did HW2 is where the work happens. Last time I built and pushed images from my own laptop. This time I'm doing the whole thing from inside the EC2 box: clone the repo there, build the images there, push them from there, and run `helm install` right on the machine that's already talking to the cluster. It cuts out a step (no copying a kubeconfig file around) and it's closer to how this would actually work outside of a class assignment.

My source lives in `swe645-h3` on GitHub — it's the same app as my working copy, just without the `virtualenv`, `node_modules`, and local SQLite file that don't belong in source control. This guide is basically my own notes on getting from "empty EC2 box with Rancher on it" to "app running behind a NodePort," written up so I (and whoever's grading this) can follow it again later.

Architecture

```
Browser
| HTTP (NodePort 30080)
▼
[frontend Pod: nginx serving the built React app]
| proxies /api/, /docs, /openapi.json (in-cluster, ClusterIP)
▼
[backend Pod: FastAPI + SQLModel + Uvicorn]
| mysql+pymysql:// (ClusterIP survey-mysql:3306)
▼
[mysql Pod: MySQL 8.4 + PersistentVolumeClaim]
```

The browser only ever talks to the frontend's NodePort. nginx reverse-proxies the API calls back to the backend Service inside the cluster, which means I never had to think about CORS in production, and the backend doesn't need a port of its own exposed to the outside world.

MySQL Storage: Static PV, Not Dynamic Provisioning

My HW2 EC2 box runs Rancher in its full RKE2 mode, not the lighter K3s distribution — `kubectl get pods -n kube-system` shows static pods like `etcd`, `kube-apiserver`, and `rke2-coredns` / `rke2-traefik` rather than K3s's single-binary footprint. That distinction matters for storage: K3s ships with a `local-path` StorageClass out of the box, but RKE2 deliberately doesn't assume any storage backend, so `kubectl get storageclass` on this cluster comes back empty.

Since MySQL needs a `PersistentVolumeClaim` and there's no default StorageClass (and no cloud CSI driver, since this is a plain EC2 box) to satisfy it dynamically, the chart provisions the volume itself: `helm/student-survey/templates/mysql-pv.yaml` defines a static, single-node `hostPath` `PersistentVolume` with no `storageClassName`, which matches the PVC's own unset `storageClassName` and binds immediately — no provisioner required. The host path is configurable via `mysql.persistence.hostPath` in `values.yaml` (defaults to `/var/lib/survey-mysql-data`).

Why `hostPath` instead of a provisioner: this is a single-node cluster with nothing else to schedule onto, so tying a volume to that one node has no real downside here. On a multi-node cluster I'd install a real provisioner (e.g. `local-path-provisioner`, since RKE2 doesn't bundle one) instead.

What's in the Repo

```
swe645-h3/
├── backend/                # FastAPI + SQLAlchemy REST API
│   ├── app/
│   │   ├── main.py        # app setup, CORS, DB-ready retry, /health
│   │   ├── database.py    # engine + session dependency
│   │   ├── models.py      # Survey SQLAlchemy table
│   │   ├── schemas.py     # SurveyCreate/Update/Read
│   │   └── routers/surveys.py # CRUD endpoints
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/              # React (Vite) SPA
│   ├── src/
│   │   ├── App.jsx, api.js, options.js
│   │   └── components/{SurveyForm,SurveyList,SurveyResultsModal}.jsx
│   ├── nginx.conf         # serves the SPA + proxies /api, /docs to backend
│   └── Dockerfile
├── helm/student-survey/   # Helm chart (Chart.yaml, values.yaml, templates/)
├── postman/student-survey.postman_collection.json
└── README.md
```

Survey Fields

Matches the assignment spec:

Field group	Details
Required fields	First name, last name, street address, city, state, zip, phone, email, survey date
Liked most about campus (<i>multi-select</i>)	Students, Location, Campus, Atmosphere, Dorm Rooms, Sports
How they became interested (<i>single choice</i>)	Friends, Television, Internet, Other
Recommendation likelihood (<i>single choice</i>)	Very Likely, Likely, Unlikely

What I'm Doing, Phase by Phase

Phase	What I'm Doing	End State
1	Connect to my HW2 EC2 box via EC2 Instance Connect, confirm Docker/Rancher/kubectl still work	Same cluster from HW2, still Ready
2	Install Helm on the EC2 instance (new tool for HW3)	<code>helm version</code> works
3	Clone the repo onto the EC2 box	Source on the server
4	Build both Docker images on EC2 and push to Docker Hub	Two images pullable by the cluster
5	Open the new NodePort in the EC2 security group	Port 30080 reachable from my browser
6	Deploy with Helm	Frontend, backend, MySQL Pods Running
7	Check that everything actually came up	App reachable at the NodePort URL
8	Run the Postman collection against the deployed backend	Full CRUD verified

Before I Start

- My HW2 EC2 instance, with Rancher and the Kubernetes cluster still running on it — I'm not rebuilding any of that
- A Docker Hub account (`lmohler`) and a GitHub account, both already set up from HW2
- `swe645-h3` already pushed to GitHub.

Why I'm doing it this way: Everything in this guide happens in a browser-based EC2 Instance Connect terminal on the EC2 box, not on my Windows machine. I don't have Docker Desktop fighting for resources with anything else, and I don't have to deal with copying a kubeconfig file to a second machine — the box already knows how to talk to its own cluster.

Phase 1 – Reconnecting to My HW2 Box

Step 1: Connect to My Instance

No key file or local terminal client needed — the browser-based EC2 Instance Connect handles it straight from the console:

1. AWS Console > EC2 > Instances > select my instance.
2. Click **Connect** > **EC2 Instance Connect** tab > **Connect**.

This opens a terminal tab directly in the browser, already logged in as `ubuntu`.

Step 2: Confirm Docker and Rancher Are Still Running

```
docker ps
```

I'm looking for the `rancher/rancher` container from HW2, status **Up**. If it's not there, `docker start <container-id>` brings it back (Docker containers don't survive a full instance stop/start unless they were started with `--restart=unless-stopped`, which mine was).

Step 3: Confirm kubectl Still Talks to the Cluster

```
kubectl get nodes
```

Should show my one node, status **Ready**. This is the same `~/.kube/config` I set up during HW2, so there's nothing new to configure here.

Phase 2 – Installing Helm

Helm wasn't part of HW2 — I used raw `kubect1 apply` against plain YAML files back then. For HW3 I'm using a Helm chart instead, so I need to actually install Helm on the box.

Step 4: Run the Official Install Script

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
```

Then check it worked:

```
helm version
```

Note: Helm automatically picks up the same `~/.kube/config` that `kubect1` already uses, so there's no separate Helm-specific configuration step.

Phase 3 – Cloning It Onto EC2

Step 5: Make Sure Git Is There

```
git --version
```

It already was, from HW2, but worth checking. If not: `sudo apt install git -y`.

Step 6: Clone the Repo

Since this repo is public and I'm not setting up any CI/CD triggers this time (that was the HW2 Jenkins pipeline, not part of this assignment), I just clone over HTTPS — no need to set up any Git credentials on this box for GitHub.

```
mkdir -p ~/repos && cd ~/repos
git clone https://github.com/lmohler/swe645-h3.git
cd swe645-h3
```

Phase 4 – Building and Pushing the Docker Images (from EC2)

This is the part that's different from how I did it the first time around — instead of `docker build` on my laptop, I'm doing it right here on the server, since Docker's already installed and this is where the image needs to end up being pulled from anyway (well, from Docker Hub, but building here saves me a round trip).

Step 7: Build the Backend Image

```
cd backend
docker build -t lmohler/survey-backend:1.0.0 .
```

Step 8: Smoke-Test It

```
docker run --rm -d -p 8000:8000 -e DATABASE_URL=sqlite:///tmp/local.db --name backend-test lmohler/survey-backend:1.0.0
curl localhost:8000/health
```

I expect `{"status": "ok"}` back. Once confirmed:

```
docker stop backend-test
```

Step 9: Build the Frontend Image

```
cd ../frontend
docker build -t lmohler/survey-frontend:1.0.0 .
```

Note: I didn't bother smoke-testing the frontend image standalone this time the way I did during local dev — without a backend Service to proxy to, it would just 502 on `/api` calls anyway, and I already know from earlier testing that the build itself works.

Step 10: Log In to Docker Hub and Push Both Images

```
docker login -u lmohler
```

(enters my Docker Hub password/token when prompted)

```
docker push lmohler/survey-backend:1.0.0
docker push lmohler/survey-frontend:1.0.0
```

Then I check hub.docker.com to confirm both repositories show tag `1.0.0`.

Tip: If I make a code change later, I bump the tag (e.g. `1.0.1`) instead of overwriting `1.0.0` — makes it obvious during `helm upgrade` which build is actually running.

Phase 5 – Opening Port 30080

Step 11: Add the Inbound Rule

1. AWS Console > EC2 > Instances > my instance > Security tab > click the security group.
2. Edit inbound rules > Add rule.

Port	Protocol	Source	Purpose
30080	TCP	<code>0.0.0.0/0</code>	Student Survey frontend NodePort

Note: If my HW2 security group already had the whole `30000-32767` NodePort range open, 30080 is already covered and I can skip this.

Phase 6 – Deploying with Helm

Step 12: Don't Leave the Default MySQL Password In

`helm/student-survey/values.yaml` ships with placeholder credentials (`changeme-root-password` , etc.). I'm overriding these at install time rather than editing the file, especially since this repo is public on GitHub.

No StorageClass setup needed: since this is RKE2 (see the storage note back in the Overview), I'd normally need to install a provisioner myself before MySQL's PVC could bind to anything. The chart already handles this — `templates/mysql-pv.yaml` provisions a static `hostPath` PV, so there's nothing extra to set up here.

Step 13: Install the Chart

```
cd ~/repos/swe645-h3/helm
helm install student-survey ./student-survey \
  --set backend.image.tag=1.0.0 \
  --set frontend.image.tag=1.0.0 \
  --set mysql.rootPassword=<my-strong-password> \
  --set mysql.password=<my-strong-password>
```

This creates the three Deployments (`survey-frontend` , `survey-backend` , `survey-mysql`), their Services, a Secret, and a PersistentVolumeClaim — all from the templates under `helm/student-survey/templates/` .

Step 14: Watch It Come Up

```
kubectl get pods --watch
```

The PVC binds to the static PV in `mysql-pv.yaml` immediately (no provisioner to wait on), so `survey-mysql` should schedule and come up right away, then `survey-backend` once it can reach MySQL (it retries the DB connection on startup — see `_create_db_and_tables_with_retry` in `backend/app/main.py`), then `survey-frontend` . `Ctrl+C` once all three say `1/1 Running` .

Phase 7 – Checking Everything Actually Came Up

Step 15: Pods and Services

```
kubectl get pods
kubectl get svc
```


All three Pods **Running**, and `survey-frontend` should be a **NodePort** Service on 30080.

Step 16: Open It in a Browser

```
http://ec2-100-52-109-4.compute-1.amazonaws.com:30080
```

Swagger docs come through the same nginx proxy at `/docs` .

If it won't load: `kubectl get pods` for `CrashLoopBackOff`, then `kubectl logs <pod-name>` . There's a troubleshooting table at the end of this doc with the issues I've actually hit.

Step 17: Rolling Out a Change Later

```
helm upgrade student-survey ./student-survey --set backend.image.tag=1.0.1
```

`helm uninstall student-survey` tears the whole thing down — MySQL PVC claim included, though whether the underlying data actually gets wiped depends on the PV's reclaim policy.

Phase 8 – Running Through Postman

Step 18: Import the Collection

In Postman: **Import** > `postman/student-survey.postman_collection.json` (I pulled this from my repo clone, or just from GitHub directly).

Step 19: Set base_url

```
http://ec2-100-52-109-4.compute-1.amazonaws.com:30080
```

nginx proxies `/api` through to the backend, so this one variable covers every request in the collection.

Step 20: Run Through Every Request

1. Health check — `{"status": "ok"}` .
2. Create a survey — `201` , full survey object back with a new `id` .
3. List all surveys — the new one shows up.
4. Get by ID — matches what I just created.
5. Update — change a field, confirm it stuck.
6. Delete — `204` , then List again to confirm it's gone.

Quick Reference Card

REST API

Method	Path	Description
GET	<code>/health</code>	Liveness/readiness check
POST	<code>/api/surveys</code>	Create a survey
GET	<code>/api/surveys</code>	List all surveys
GET	<code>/api/surveys/{id}</code>	Get one survey
PUT	<code>/api/surveys/{id}</code>	Update a survey
DELETE	<code>/api/surveys/{id}</code>	Delete a survey

Full interactive docs (request/response schemas) come from FastAPI automatically at `/docs` .

Helm Commands I Actually Use

Command	What it does
<code>helm install <release> <chart-dir></code>	Deploy the chart the first time
<code>helm upgrade <release> <chart-dir></code>	Push changed values/images to an existing release

<code>helm uninstall <release></code>	Remove everything the chart created
<code>helm list</code>	Show installed releases
<code>helm status <release></code>	Show a release's current state and notes
<code>helm get values <release></code>	Show what values are actually applied

Git Commands I Actually Use

Command	What it does
<code>git status</code>	What's changed or untracked
<code>git add .</code>	Stage everything changed
<code>git commit -m "message"</code>	Save a snapshot
<code>git push</code>	Send commits to GitHub
<code>git pull</code>	Grab the latest from GitHub (e.g. onto the EC2 clone)
<code>git log --oneline</code>	Recent commit history

Key Files

File	Purpose
<code>backend/Dockerfile</code>	Builds the FastAPI/SQLModel image
<code>frontend/Dockerfile</code>	Multi-stage build: Vite build → nginx serve
<code>frontend/nginx.conf</code>	Serves the SPA + proxies <code>/api</code> , <code>/docs</code> to the backend
<code>helm/student-survey/values.yaml</code>	Image repos/tags, replica counts, NodePort, MySQL credentials
<code>helm/student-survey/templates/*.yaml</code>	Deployments, Services, Secret, PVC + static hostPath PV for frontend/backend/mysql
<code>postman/student-survey.postman_collection.json</code>	Health check + full CRUD requests, driven by a <code>base_url</code> variable

Troubleshooting

Problem	What I'd Check
App loads but survey data never saves	<code>kubectl logs deploy/survey-backend</code> for a DB connection error; confirm <code>survey-mysql</code> is Running first — the backend only retries so many times on startup
<code>survey-backend</code> stuck in CrashLoopBackOff	Usually means MySQL wasn't ready in time. <code>kubectl delete pod <backend-pod></code> forces a restart once <code>survey-mysql</code> is confirmed Running
Browser shows 502 on <code>/api</code> calls	Frontend nginx can't reach <code>survey-backend:8000</code> — check the backend Service and Pod are both healthy
NodePort 30080 not reachable from my browser	Security group inbound rule for TCP 30080 from <code>0.0.0.0/0</code> (Phase 5)
<code>helm install</code> fails: "cannot re-use a name"	A release with that name already exists — <code>helm uninstall student-survey</code> first, or pick a new release name
Image pull errors in <code>kubectl describe pod</code>	Confirm the image actually got pushed and that <code>values.yaml</code> 's repository/tag match what I pushed
MySQL Pod won't start / PVC pending	The chart's own static PV (<code>mysql-pv.yaml</code>) should make this a non-issue, but if it still happens: <code>kubectl get pv,pvc</code> to confirm <code>survey-mysql-pv</code> exists and is <code>Bound</code> , and <code>kubectl get storageclass</code> to double check nothing's now shadowing it with a real default StorageClass
Postman requests fail with connection refused	Double check <code>base_url</code> actually matches the NodePort URL I'm testing against
<code>docker build</code> or <code>docker push</code> hangs on EC2	Check the instance still has internet access and enough disk space (<code>df -h</code>) — image layers add up
EC2 public DNS changed since last time	No Elastic IP attached — AWS Academy sessions reassign it on restart. I check the console for the current one